

>_ SESSION 04 • WEB DEVELOPMENT

Layout with Flexbox

The flex model, justifying and aligning content, wrapping and sizing, and the patterns you'll use every day.



>_ Vasken Sarkis • Layout with Flexbox

Today's session

- 01 The flexbox model — containers, items, and the two axes
- 02 Justify & align — positioning items on each axis
- 03 Wrapping & sizing — flex-wrap, grow, shrink, basis
- 04 Common layout patterns — navbars and card rows



ALSO TODAY

Lab: build a responsive navigation bar and a 3-card feature section, both with Flexbox.

01

The Flexbox Model

A one-dimensional layout system — one row or one column of items at a time.

Flex containers, flex items

- 01 Set `display: flex` on a **container**.
- 02 Its direct children instantly become **flex items** — no other markup change needed.
- 03 Container properties (`justify-content`) and item properties (`flex`) are never mixed up once you know which side they live on.



KEY IDEA

Reading a flex property? First ask: is this describing the box, or one of the things inside it?

>_ THE FLEXBOX MODEL

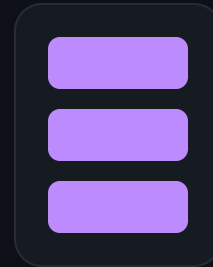
Two axes, one direction

`flex-direction: row`



main axis →

`flex-direction: column`



main axis ↓

The **main axis** runs along `flex-direction`. The **cross axis** always runs perpendicular to it.

02

Justify & Align

Two properties, two axes — one for spacing items out, one for positioning them across.

>_ JUSTIFY & ALIGN

justify-content works the main axis

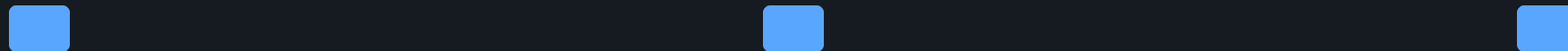
flex-start



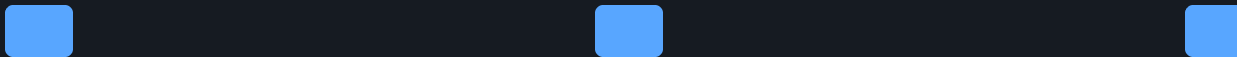
center



space-between



space-around



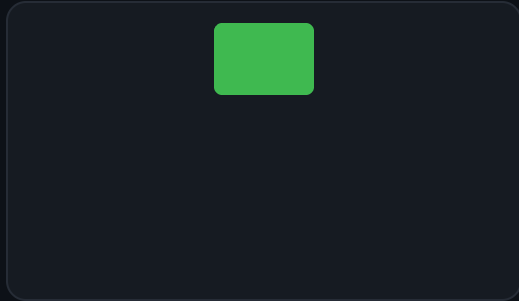
TIP

space-between is the workhorse for navbars: logo pinned left, links pinned right, nothing in the middle.

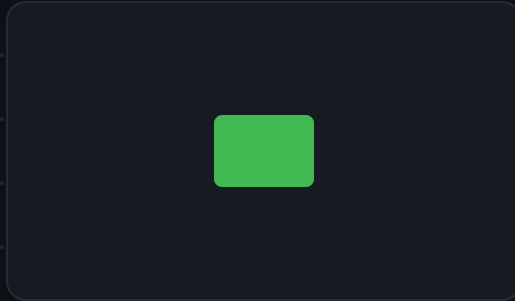
>_ JUSTIFY & ALIGN

align-items works the cross axis

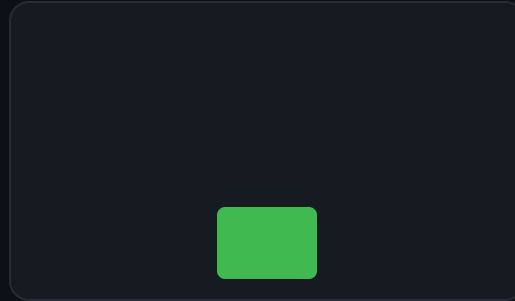
flex-start



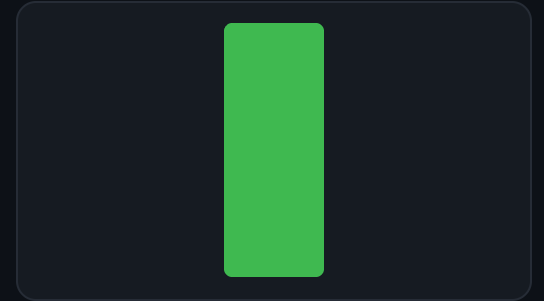
center



flex-end



stretch



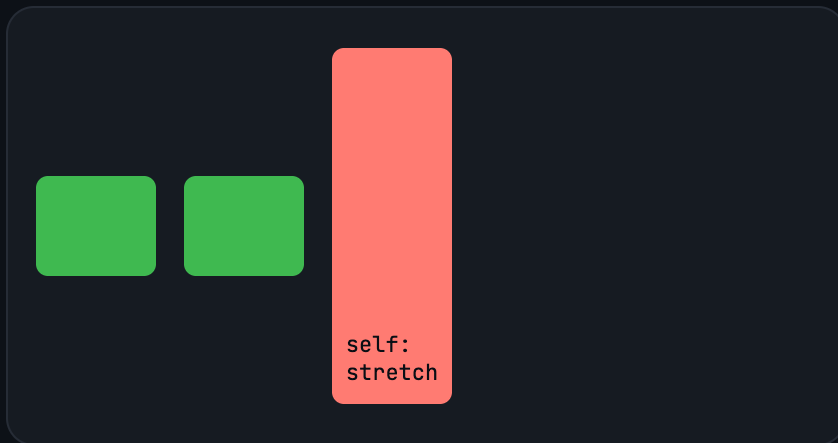
GOTCHA

stretch is the default — items fill the container's cross-axis size unless you give them a height.

> _ JUSTIFY & ALIGN

align-self overrides one item

`align-items: center`



NOTE

Every flex item can set its own `align-self`, breaking away from the container's `align-items`.



TIP

Useful for a single sidebar card, a highlighted item, or a full-height divider.

03

Wrapping & Sizing

What happens when items don't fit on one line, and how each item decides how much room to take.

flex-wrap lets items drop to a new line

cards.css

```
1 .row {  
2   display: flex;  
3   flex-wrap: wrap;  
4   gap: 20px;  
5 }
```



NOTE

Without `flex-wrap`, items shrink forever to stay on one line — they never move down.



TIP

`flex: 1 1 260px` reads as "grow, shrink, but prefer 260px" — the basis for a responsive card row.

The flex shorthand: grow, shrink, basis

flex-grow

how much extra space to take

flex-shrink

how much to give up when tight

flex-basis

the starting size, before growing/shrinking

```
.card { flex: 1 1 260px; }
```

Shorthand order is always **grow shrink basis**. `flex: 1` is shorthand for `1 1 0`.

Long content can still overflow

```
fix.css

.item {
  flex: 1 1 0;
  min-width: 0;
}
```



GOTCHA

Flex items default to `min-width: auto` — long unbreakable text or a wide image can force an item past its basis and overflow the row.



TIP

Setting `min-width: 0` lets the item actually shrink as small as `flex-shrink` intends.

04

Common Layout Patterns

Two patterns cover a huge share of real-world flexbox use:
navbars and card rows.

>_ COMMON LAYOUT PATTERNS

A navbar is three flex rules

```
1 .navbar {  
2   display: flex;  
3   align-items: center;  
4   justify-content: space-between;  
5 }  
6 .nav-links {  
7   display: flex;  
8   gap: 24px;  
9 }
```

Brand

Home About Contact

↑ what today's lab builds

>_ Vasken Sarkis · Layout with Flexbox

A card row wraps and grows together

cards.css

```
1 .feature-row {  
2   display: flex;  
3   flex-wrap: wrap;  
4   gap: 24px;  
5 }  
6 .feature-card {  
7   flex: 1 1 280px;  
8 }
```

Card 1

Card 2

Card 3

gap replaces margin hacks

row.css

```
1 .row-old > * + * {  
2   margin-left: 16px;  
3 }  
4 .row-new {  
5   display: flex;  
6   gap: 16px;  
7 }
```



KEY IDEA

gap spaces children evenly without a fragile `:not(:last-child)` selector.



TIP

Works the same way in Grid — one property, both layout systems.

Build a navbar and a feature section

Part 1 — Navigation bar

1. Logo on the left, links on the right
2. `justify-content: space-between`
3. Links spaced with `gap`, not margins

Part 2 — 3-card feature section

1. Three cards, equal width, wrapping on narrow screens
2. `flex-wrap: wrap + flex: 1 1 260px`
3. Consistent spacing with `gap`